

(Agentic) Software Engineering

BU CAS CS 413 Fall 2026

(Syllabus)

- **Semester** Fall, 2026

- **Teaching Staff**

- Instructor – Hongwei Xi
- Teaching Assistant – Victor Wang
 - * Office Hours: TBA
 - * Location: CDS 1013, e-mail: hwxi@cs.bu.edu

- **Lecture Times:** TR 12:30-1:45PM

- **Classroom:** CAS 211

- **Reference Books:**

- Ian Sommerville, Software Engineering, **9th** edition, Pearson/Addison-Wesley, 2011.
 - * ISBN: 978-0-13-703515-1
 - * *The textbook provides the conceptual foundation for the course. Some topics involving current development practices will be supplemented with additional readings and examples.*
- Robert C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship, Prentice Hall, 2008.
 - * ISBN: 978-0-13-235088-4
 - * *This book supplements Sommerville by focusing more closely on the design and implementation of maintainable source code.*
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
 - * ISBN: 978-0-201-63361-0
 - * *This book presents a vocabulary and catalog of recurring solutions to object-oriented software design problems.*

- **Homepage:** <https://hwxi.github.io/TEACHING/CS413/2026Fall>

- **Description:** This course introduces the principles and practices used in the engineering of medium/large-scale software systems. It focuses on software development as a disciplined and collaborative engineering activity rather than simply the construction of programs.

Primary topics include software processes, agile development, requirements engineering, system modeling, architectural and object-oriented design, software testing, software evolution,

dependability and security, software reuse, project management, quality management, and configuration management.

A major component of the course is a semester-long team software project (involving the construction of a compiler for a small functional programming language). Students apply the techniques discussed in class to the specification, design, implementation, testing, and evolution of a nontrivial software system.

The course uses Ian Sommerville's Software Engineering, 9th edition, as its primary text while supplementing the book with selected contemporary software engineering practices.

- **Course Philosophy**

Software engineering is fundamentally concerned with *complexity, change, quality, and collaboration*.

Programming asks:

- How can we construct a program that performs a particular computation?

Software engineering asks a broader set of questions:

- What system should we build?
- What do its users actually need?
- How should those requirements be specified?
- How should the system be structured?
- How can multiple developers work on it safely?
- How can we establish confidence that it works?
- How do we manage changing requirements?
- How do we manage defects and failures?
- How do we release and maintain the system?
- How can another developer understand and modify it years later?

The central goal of this course is therefore not merely to make students better programmers. It is to teach students to approach software development as an **engineering discipline** in which requirements, design, evidence, tradeoffs, process, and communication are as important as writing code.

- **Prerequisites:** This course is designed for students who already have an immediate level of proficiency in Python. You are expected to be familiar with a text editor (e.g., vim, emacs, vscode).

- **Exams:**

- First midterm exam: TBA
- Second midterm exam: TBA
- The final exam date is yet to be announced.

- **Grades** The final score is calculated using the following formula.

$$\text{final score} = 30\% \cdot (\text{homework}) + 30\% \cdot (\text{midterm}) + 30\% \cdot (\text{final}) + 10\% \cdot (\text{participation})$$

The final letter grade is calculated as follows.

- **A:** final score is 90% or above
 - **B:** final score is 80% or above
 - **C:** final score is 70% or above
 - **D:** final score is 60% or above
- **Homework Submission:** Homework assignments are to be submitted via the Gradescope system.
 - **Attendance:** It is expected that you will attend the lectures.
 - **Use of Generative AI** Generative AI systems are increasingly used as software development tools. Students may use such systems when explicitly permitted for an assignment or project. Permitted uses may include:
 - Explaining unfamiliar code
 - Brainstorming alternative designs
 - Generating candidate test cases
 - Assisting with debugging
 - Generating routine implementation code
 - Reviewing code or documentation
 - Improving documentation

Students remain responsible for everything they submit. In particular, students must be able to:

- Explain submitted code.
- Explain important design decisions.
- Verify generated code.
- Test generated code.
- Identify incorrect or inappropriate generated solutions.
- Consider security and maintainability implications.
- Properly acknowledge AI assistance when required.

The fact that code was generated by an AI system is not evidence that the code is correct.

The instructor may ask a student to explain any submitted code or design. Inability to explain one's own submission may affect the student's grade and, where appropriate, raise an academic-integrity concern.

- **Academic Integrity:** We adhere strictly to the standard BU guidelines for academic integrity. For this course, it is perfectly acceptable for you to discuss the general concepts and principles behind an assignment with other students. However, it is not proper, without prior authorization of the instructor, to arrive at collective solutions. In such a case, each student is expected to develop, write up and hand in an individual solution and, in doing so, gain a sufficient understanding of the problem so as to be able to explain it adequately to the instructor. Under *no* circumstances should a student copy, partly or wholly, the completed solution of another student. If one makes substantial use of certain code that is not written by oneself, then the person must explicitly mention the source of the involved code.